A Spatio-Temporal Expert Prefetching Framework for Efficient MoE-based LLM Inference

Yingnan Zhao
George Washington University
Washington, DC, USA
yzhao96@gwu.edu

Razvan Bunescu
University of North Carolina at Charlotte
Charlotte, NC, USA
razvan.bunescu@charlotte.edu

Ahmed Louri
George Washington University
Washington, DC, USA
louri@gwu.edu

Avinash Karanth
Ohio University
Athens, OH, USA
karanth@ohio.edu

Ke Wang
University of North Carolina at Charlotte
Charlotte, NC, USA
ke.wang@charlotte.edu

arXiv:2606.15453v1 [cs.AR] 13 Jun 2026

Abstract

Mixture-of-Experts (MoE) based large language models (LLMs), such as Qwen and DeepSeek, have recently emerged as an effective approach to improving model capacity without proportionally increasing computational cost. By replacing the conventional feed-forward network in dense LLMs with a set of experts and activating only a subset of them for each input token, MoE models significantly increase the total number of parameters while keeping the per-token computation relatively manageable. However, this dynamic and irregular expert activation pattern also introduces substantial expert loading overhead during inference, since the required experts must be fetched on demand according to token-dependent routing results. As a result, expert loading latency becomes a major source of performance and energy inefficiency. Existing approaches attempt to address this challenge through methodologies such as adaptive routing and pre-gating. These methods often suffer from inference accuracy degradation or introduce additional training overhead, limiting their practical benefits.

To this end, we first perform a comprehensive analysis of expert selection behavior in various MoE-based LLMs and applications, including language understanding and code generation. Our analysis reveals that, within each application domain, expert requests exhibit strong correlation across both adjacent MoE layers and consecutive decoding tokens, making future expert activations predictable. Based on this insight, we propose ST-MoE, a spatio-temporal expert prefetching framework that proactively stages experts ahead of use to overlap expert loading with ongoing computation. ST-MoE combines a lightweight runtime prediction mechanism that preserves the original routing behavior with a reconfigurable hardware design that efficiently supports dynamic expert prefetching. The combined effect of the prediction mechanism with the supporting hardware significantly improves MoE inference performance and energy efficiency while preserving model inference accuracy. Evaluation results on real-world applications show that ST-MoE achieves 85% expert prediction accuracy and delivers average speedups of 2.5x, 2.2x, and 1.5x, as well as average energy efficiency improvements of 2.5x, 1.8x, and 2.0x, compared to GPU, Adap-Gating, and Pre-gated MoE, respectively.

1 Introduction

Large Language Models (LLMs) have achieved remarkable success across a wide range of applications, such as strategic reasoning [47, 48], language understanding [21, 29], code generation [11, 17], and multimodal intelligence [16, 41, 46]. A key driver of this progress has been the continued scaling of model capacity, which has consistently improved LLM capability across diverse applications. However, scaling dense LLMs often incurs rapid increase in computation and memory costs, making it increasingly challenging for efficient deployment. To address this issue, the Mixture-of-Experts (MoE) technique is now widely used in both frontier and open-source LLMs [2, 20, 44]. By replacing the conventional dense feed-forward network with a set of experts and activating only a subset of them for each input token, MoE-based LLMs increase the total number of parameters while significantly decreasing the number of parameters that are actively used per token. However, the decrease in the number of parameters comes at the cost of a more complex computational path during inference.

For each token, the gating function dynamically selects only a small subset of experts, making the computation path highly input dependent. The experts required by one token may differ from those required by the next token or by the next MoE layer, making expert requests difficult to schedule statically. Since each expert contains a large number of parameters, keeping all experts on-chip is impractical due to limited buffer capacity. Instead, the required expert weights must be fetched on demand from off-chip memory, which incurs substantial data movement and memory access latency. This challenge becomes even more critical during the decoding phase when tokens are generated sequentially. Consequently, any delay in loading the required experts can stall subsequent computation, reduce hardware utilization, and increase energy consumption. Therefore, although MoE lowers the computational cost for LLMs, inefficient expert loading can significantly limit the performance and energy efficiency benefits.

To address the inference challenges of MoE-based LLMs, prior work has explored several approaches, including adaptive routing, expert prefetching, and expert caching. For example, Adap-gating [24] improves inference efficiency by dynamically reducing the number of activated experts, while Pre-gated MoE [19] introduces an additional trained pre-gating function together with

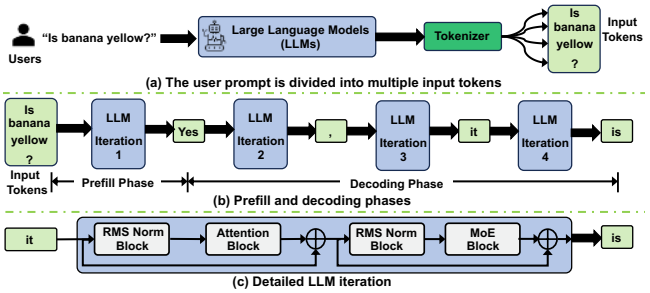

Figure 1: (a) The user prompt is tokenized into multiple input tokens by the LLM tokenizer. (b) LLM inference consists of two phases: prefill and decoding. (c) One LLM iteration consists of three types of blocks: Root Mean Square (RMS) Normalization, attention, and Mixture-of-Experts (MoE). DeepSeek V2 is used here as a representative LLM model.

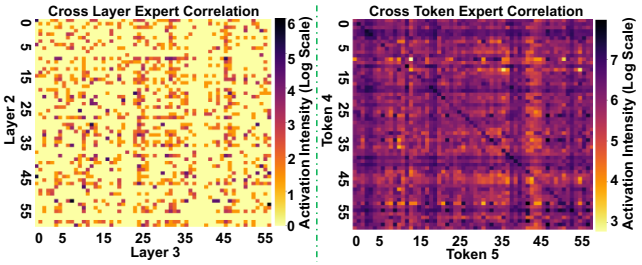

Figure 2: (a) Spatial relationship of different experts across consecutive MoE layers for the same token. (b) Temporal relationship of consecutive decoding tokens within the same MoE layer.

CPU-GPU co-design to proactively transfer the experts required by the next MoE block. Despite their advantages, these approaches still face multiple limitations for effective deployment. Adap-gating alters the routing behavior learned under the original training policy, which can lead to significant inference accuracy degradation, whereas Pre-gated MoE depends on an additional trained component, which imposes extra training overhead and deployment complexity. More broadly, existing methods fail to fully exploit structured expert activation patterns that naturally emerge during decoding. As a result, they leave significant room to improve MoE inference efficiency in a practical, adaptive, and accuracy-preserving manner.

To this end, we conduct a comprehensive empirical analysis of expert selection behavior in representative MoE-based LLMs across diverse applications. Our study reveals two important types of correlations in the MoE mechanism when used with LLMs: expert selections are strongly correlated across adjacent MoE layers (spatial) and between consecutive decoded tokens (temporal), making expert requests amenable to predictability at runtime. Building on this critical insight, we propose ST-MoE, a spatial-temporal expert prefetching framework that proactively stages experts before they are needed to overlap expert loading latency with ongoing computation. ST-MoE integrates a lightweight dynamic prediction mechanism with a reconfigurable hardware architecture to enable efficient expert prefetching without altering the original routing outcome or requiring additional training. The main contributions of this work are as follows:

- **Comprehensive Expert Correlation Analysis:** We conduct a comprehensive analysis of expert selection behavior in representative MoE-based LLMs across several applications, and show that expert activations exhibit strong cross-layer and cross-token correlation during decoding. To the best of our knowledge, this is the first systematic documentation of these cross-layer and cross-token correlations.
- **Lightweight Expert Prediction Strategy:** We propose a lightweight dynamic expert prediction strategy that leverages a cross-layer correlation table and a history table to predict future expert requests without modifying the original MoE model or introducing additional training overhead.
- **Prediction-Guided Reconfigurable Hardware:** We design a reconfigurable hardware architecture that supports prediction-guided expert prefetching, flexible expert staging, and pipelined execution, effectively hiding expert loading latency and improving both performance and energy efficiency.

Evaluation results on real-world applications show that ST-MoE achieves 85% expert prediction accuracy and delivers average speedups of 2.5x, 2.2x, and 1.5x, as well as average energy efficiency improvements of 2.5x, 1.8x, and 2.0x, compared to GPU, Adap-Gating, and Pre-gated MoE, respectively.

2 LLM Inference and Mixture-of-Experts

Given a user prompt, an LLM first uses a pre-determined tokenizer to convert the prompt into a sequence of input tokens for subsequent inference, as illustrated in Fig. 1(a). Typically, LLM inference consists of two key phases: *prefill* and *decoding*, as shown in Fig. 1(b). During the prefill phase (LLM iteration 1), the model processes all input tokens in parallel to construct intermediate states. During the decoding phase, in contrast, output tokens are generated sequentially, and each newly generated token depends on the previously generated tokens [23, 42]. Within each LLM iteration, at each block in the Transformer decoder, the model uses a Mixture-of-Experts (MoE) architecture comprised of N experts implemented as feed-forward networks. Modern LLMs rely on a large and increasing number of experts, typically ranging from 8 to over 256 experts per layer [9, 18, 26, 27, 33, 36]. The MoE architecture also contains a gating network that is specified through N vectors of parameters $\mathbf{w}_j$, one vector for each expert $1 \leq j \leq N$. Given an arbitrary token with hidden representation $\mathbf{x}$, the gating network computes for each expert a gating score $z_j = \mathbf{w}_j^T \mathbf{x}$ that is then mapped into a gating probability s_j by applying a softmax transform. The experts corresponding to the top K gating probabilities are then selected to be used with the input $\mathbf{x}$, and their outputs are summed up, using their gating probabilities as weights [8].

3 Spatio-Temporal Correlations in MoE

A key challenge in MoE-based LLM inference is that expert requests are generated dynamically at runtime. However, our analysis shows that, within the same application domain, expert selections are not

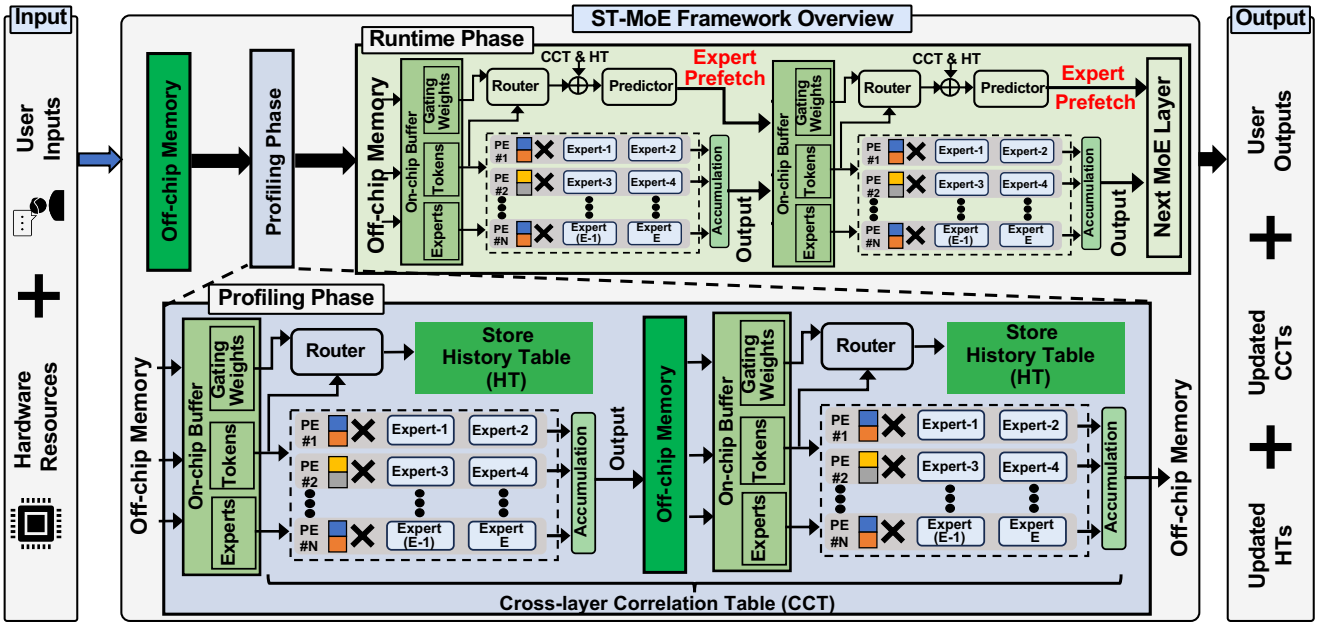

Figure 3: Overview of the proposed ST-MoE framework. The framework consists of two phases: profiling and runtime. The profiling phase constructs the History Table (HT) for each MoE layer and the Cross-layer Correlation Table (CCT), while the runtime phase uses these tables to dynamically predict the experts likely to be activated.

independent. Instead, they exhibit strong correlation across consecutive layers and between consecutive decoded tokens, which creates an opportunity to predict expert requests and enable effective expert prefetching, improving performance and energy efficiency.

3.1 Cross-Layer Expert Correlation

Upon collecting statistics from multiple MoE-based LLMs, we observe that expert selections for the same token are correlated across consecutive MoE layers. This behavior can be explained by the fact that transformer layers operate on progressively refined representations of the same token while preserving its high-level semantic characteristics, which causes routing decisions in neighboring layers to be statistically dependent. We first analyze the DeepSeek model [7, 26] on the CNN Daily Mail dataset [4, 15] using an article summarization workload. Fig. 2(a) shows the expert co-activation heatmap between two consecutive MoE layers, Layer 2 and Layer 3, based on 100,000 input tokens. We further perform a chi-squared test on cross layer expert selections, and the resulting p values are consistently below 0.01, confirming that expert activations across consecutive layers are not independent. We observe similar strong correlations across different models, including Qwen [1, 39] and Mixtral [20], and across different datasets, including HumanEval [3, 5, 31] and MATH [13, 25].

3.2 Consecutive-Token Expert Correlation

Expert selections within the same MoE layer are also observed to be correlated between consecutive decoded tokens. This correlation arises because autoregressive decoding generates each token based on the previously generated context, causing neighboring tokens to exhibit similar semantic dependencies and hidden representations. Consequently, consecutive decoding tokens often trigger overlapping expert selections. We use the same experimental setup as in Section 3.1, including the models, datasets, and applications. Fig. 2(b) shows the expert co-activation heatmap for two consecutive decoding tokens within the same MoE layer. We further compare the measured overlap in expert selections between consecutive decoded tokens against a random routing baseline. For an MoE layer with N experts, where each token activates K experts, the expected overlap under independent random routing is $E(N) = K^2/N$. Across multiple LLMs and datasets, the observed overlap consistently exceeds this baseline by nearly 2 times, confirming that expert selections across consecutive decoding tokens are correlated rather than independent.

4 ST-MoE Framework Design

Our observations show that expert selections exhibit cross-layer and cross-token correlation during decoding, indicating that future expert requests are predictable from prior routing behavior. Motivated by this, we propose an application-aware dynamic expert prefetching framework that combines algorithm and hardware co-design to proactively stage experts before they are needed, thereby reducing expert loading latency and improving energy efficiency without compromising model inference accuracy. In this section, we first present the overview of the proposed framework design in Sec. 4.1. We then describe the algorithm-level design in Sec. 4.2, which exploits both cross-layer and cross-token correlation to predict the experts required by each MoE layer dynamically. Finally, we present the reconfigurable hardware architecture in Sec. 4.3,

which efficiently supports dynamic expert prefetching to improve performance and energy efficiency.

4.1 Framework Overview

Fig. 3 illustrates the overall workflow of the proposed ST-MoE framework, which consists of two main phases: *Profiling* and *Runtime*. In the profiling phase, ST-MoE uses a small set of input tokens to capture per-layer expert selection behavior and cross-layer expert correlation. Then, ST-MoE records the resulting information in a History Table (HT) and a Cross layer Correlation Table (CCT), respectively. These tables are used during the runtime phase to guide dynamic expert prediction and prefetching for each MoE layer. During the runtime phase, while the current MoE layer is being executed, ST-MoE uses the gating network's result of the current layer, together with the HT and CCT, to predict the experts likely to be activated in the next MoE layer. The predicted expert indices are then sent to the memory interface of the hardware platform, which proactively prefetches the corresponding experts in parallel with computation, as shown in Fig. 3. For clarity, the attention block is omitted from the figure, while the detailed workflow and the overlap between expert prefetching and computation are described later in Sec. 4.3.4. When execution reaches the MoE layer, ST-MoE compares the prefetched experts with the results of the gating network, updates the corresponding HT and CCT entries, and retrieves any missing experts caused by misprediction without compromising inference accuracy.

4.2 Dynamic Prediction Strategy

To predict future activated experts for input tokens in each MoE layer, ST-MoE introduces a dynamic prediction strategy that jointly leverages both spatial and temporal correlation across adjacent MoE layers and consecutive decoding tokens, thereby enabling expert prefetching ahead of execution. To realize this strategy, ST-MoE introduces two key data structures: the Cross-layer Correlation Table (CCT) to capture cross-layer expert correlations and the History Table (HT) to record expert selection history for the previous decoding token. The prediction workflow consists of three steps: table construction, expert prediction, and table update. We first describe the design and initialization of the CCT and HT, and then explain how they are used and updated during execution.

Cross-layer Correlation Table (CCT): The CCT is designed to capture correlations among selected experts across adjacent MoE layers, as outlined in Algorithm 1. For each expert in the current MoE layer, the CCT stores the most strongly correlated experts in the next MoE layer, providing cross-layer guidance for future expert prediction. To build the CCT, ST-MoE first profiles a small set of input tokens and records expert co-activation patterns for each pair of adjacent layers (Line 7-12). Based on these statistics, it constructs a correlation heatmap for each adjacent layer pair, where each entry indicates how often an expert in the current layer is associated with an expert in the next layer. ST-MoE then selects the $Top-K$ correlated experts in the next layer for each expert in the current layer and stores them in the corresponding CCT entry, where K is set to match the number of activated experts in the target LLM (Line 13-15). In this way, the CCT transforms dense cross-layer correlation statistics into a compact lookup structure that enables efficient expert prediction in the Runtime phase.

Algorithm 1 Build Cross-layer Correlation Table (CCT)

1: **Input:** P: number of tokens for the profiling phase, E: number of experts in each MoE layer, K: number of candidate experts per entry.
2: **Output:** CCT$[E][2K]$: cross-layer correlation table.
3: // 11: strongly preferred, 10: weakly preferred
4: // 01: weakly not preferred, 00: strongly not preferred
5: **Initialize:**
6: Co-matrix$[E][E] \leftarrow 0$
7: **for** $s = 1$ to P **do**
8: $E_i \leftarrow$ experts selected at layer i
9: $F_i \leftarrow$ experts selected at layer $(i+1)$
10: **for all** $e \in E_i$ **do**
11: **for all** $f \in F_i$ **do**
12: Co-matrix$[e][f] \leftarrow$ Co-matrix$[e][f] + 1$
13: **for** $e = 1$ to E **do**
14: CCT$[e][*] \leftarrow$ Top-K experts in Co-matrix$[e][f]$
15: CCT$[e][*]_{ConfScore} \leftarrow 10$
16: **return** CCT

As expert selection patterns evolve during inference processing, the CCT must be updated dynamically to maintain accurate cross-layer correlation. Inspired by conventional branch predictor design, ST-MoE associates each CCT entry with a 2-bit confidence score for every stored correlated expert. Accordingly, each score has four possible states: *00* denotes strongly not preferred, *01* denotes not preferred, *10* denotes preferred, and *11* denotes strongly preferred. All entries are initialized to *10*, so each stored expert is initially treated as a preferred candidate (Line 15 in Algorithm 1). During execution, the confidence score is updated based on whether the corresponding expert matches the actual result of the gating network. Once the score falls below the preferred region, defined as 0 in our design, the associated expert becomes eligible for replacement by another expert not currently stored in the CCT. The replacement policy is described later as part of the overall workflow.

History Table (HT): The HT captures correlations across consecutive decoding tokens within the same MoE layer. For each MoE layer, the HT stores the actual $Top-K$ experts selected by the gating network for the previous decoding token. As discussed in Sec. 3, consecutive tokens in autoregressive decoding often exhibit correlated routing behavior. Therefore, the expert selections of the previous token provide useful temporal information for predicting the experts required by the current token. Similar to the CCT, each HT stores K experts, where K follows the same Top-K routing rule as the target LLM model. Specifically, the HT records the actual experts selected by the gating network for the previous decoding token in each MoE layer. For consistency with the CCT-based prediction logic, each stored expert is also associated with a 2-bit confidence score. Unlike the CCT, however, these scores are not updated during execution and are initialized to *10*. After each token is processed, the HT is overwritten with the latest Top-K routing results, so that it always reflects the expert selection outcome of the immediately preceding token. Therefore, the HT provides a lightweight and reliable representation of cross-token correlation.

Algorithm 2 Expert Predict Function

1: **Input:** t: current token index, E_i: experts selected at layer i for token t, CCT: cross-layer correlation table, HT: history table
2: **Output:** PrefetchSet: experts to prefetch for layer $(i+1)$
3: **Initialize**
4: ConfScore$[E] \leftarrow 0$ for all experts E
5: **for all** expert $e \in E_i$ **do**
6: **for all** candidate expert $f \in$ CCT$[e]$ **do**
7: ConfScore$[f] \leftarrow$ ConfScore$[f]$ + CCT$[e][f]$
8: **for all** expert $h \in$ HT **do**
9: ConfScore$[h] \leftarrow 1$
10: **for all** expert $h \in$ HT **do**
11: ConfScore$[h] \leftarrow$ ConfScore$[h]$ + 1
12: PrefetchSet $\leftarrow \{ f \mid$ ConfScore$[f] \geq 2 \}$
13: **return** PrefetchSet

Prediction Workflow: Based on the CCT and HT, ST-MoE performs dynamic expert prediction and table maintenance using two functions: Expert Predict and Table Update, as detailed in Algorithm-2 and Algorithm-3, respectively. In the predict function, once the results of the gating network for the current MoE layer become available, ST-MoE predicts the expert requests for the next MoE layer by jointly using the CCT and HT. Specifically, the selected experts of the current layer are used to index the corresponding CCT entries, producing a set of spatially correlated candidate experts for the next layer (Line 5-7). Subsequently, ST-MoE accesses the HT entry of the next layer to obtain temporally correlated expert candidates derived from the previous decoding token (Line 10-11). The candidate experts from the two sources are merged, and their confidence scores are aggregated. If an expert appears in both the CCT and HT, its final confidence score is computed by combining the two scores, as shown in Eq. 1; otherwise, its score remains unchanged. Finally, ST-MoE selects all experts whose confidence scores are no smaller than the threshold as the predicted expert set for the next MoE layer and forwards their indices to the memory interface of the hardware platform for proactive prefetching. In our prediction algorithm, this threshold is set to 2, corresponding to the 2-bit confidence state (10).

$$\text{Score}(e) = \begin{cases} \text{CCT}(e) + \text{HT}(e), & \text{if } e \in \text{CCT and } e \in \text{HT}, \\ \text{CCT}(e), & \text{if } e \in \text{CCT and } e \notin \text{HT}, \\ \text{HT}(e), & \text{if } e \notin \text{CCT and } e \in \text{HT}. \end{cases} \quad (1)$$

For the Table Update function, as shown in Algorithm-3, once the actual result of the gating network for the predicted MoE layer becomes available, ST-MoE compares it with the predicted expert set and updates the corresponding CCT and HT entries accordingly. For the CCT, if a predicted expert is confirmed by the actual routing result, its confidence score is increased by 1 (Line 7), with the maximum value capped at *11*, which denotes the strongly preferred state. Otherwise, its confidence score is decreased by 1 (Line 10). If the confidence score falls below *0* in our design, an expert replacement procedure is triggered. In this case, the corresponding expert is replaced with a new candidate expert (Line 12-14), and the confidence score of the new expert is initialized to *10*, indicating the preferred state. Meanwhile, the HT is overwritten with the latest *Top-K* expert selection results so that it always reflects the routing outcome of the immediately preceding token. Through this online update process, ST-MoE continuously adapts the CCT and HT to evolving expert selection patterns, thereby improving the prediction accuracy of subsequent MoE layers.

Algorithm 3 Table Update Function

1: **Input:** E_i: experts selected at layer i, F_j: experts selected at layer $(i+1)$, CCT: cross-layer correlation table
2: **Output:** Updated CCT
3: **for all** expert $e \in E_i$ **do**
4: PrefetchSet $\leftarrow$ experts stored in CCT$[e][*]$
5: **for all** $(f,$ ConfScore$) \in$ CCT$[e][*]$ **do**
6: **if** $f \in F_j$ **then**
7: ConfScore $\leftarrow \min($ConfScore + 1, 3$)$
8: **else**
9: **if** ConfScore > 0 **then**
10: ConfScore $\leftarrow$ ConfScore $- 1$
11: **else**
12: $g \leftarrow$ any expert in $(F_j \setminus$ PrefetchSet$)$
13: $f \leftarrow g$
14: ConfScore $\leftarrow 10$
15: **return** CCT

4.3 Reconfigurable Architecture Design

To enable the proposed dynamic expert prediction strategy in MoE inference, we implement a reconfigurable architecture that integrates expert prefetching support, flexible compute resources, and an adaptive interconnection network. In ST-MoE, reconfigurability is exposed through three coordinated mechanisms: expert-to-processing element (PE) mapping, permutation-network-based data routing, and PE-level dataflow configuration. These mechanisms are driven by prediction-guided expert availability, where the predicted expert set determines which expert weights are proactively staged on chip, while the actual gating results determine how the staged experts are mapped, routed, and executed. By coupling expert prefetching with runtime mapping, configurable data routing, and PE-level dataflow adaptation, the architecture overlaps expert transfer with computation while maintaining energy efficiency. In the following, we first introduce an overview of the architecture, followed by the prefetching hardware, the reconfigurable compute engine and interconnection design, and the overall pipelined execution workflow.

4.3.1 ***Architecture Overview:*** Fig. 4 illustrates the overall architecture of the proposed ST-MoE hardware platform, which is designed to support prediction-guided expert prefetching for efficient MoE-based LLM inference. The architecture consists of multiple key hardware components, including the Expert Prediction Unit (EPU), Expert Mapping Unit (EMU), Router, Expert and KV Buffer, Activation Buffer, Processing Elements (PEs), Permutation Network, and Activation Unit. Specifically, during execution, the router performs the gating network and generates the actual expert selection pattern for the current input token. The EPU uses this routing information, together with the CCT and HT, to predict the experts likely to be required by the subsequent MoE layer. The

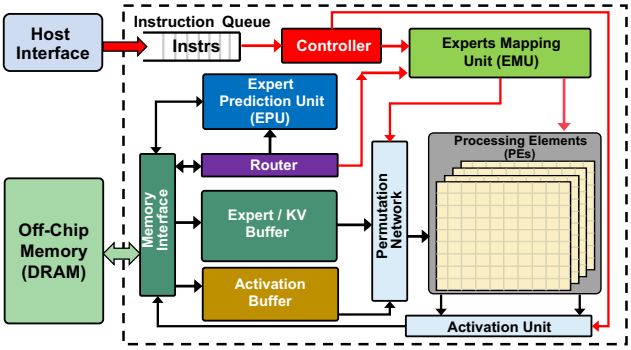

Figure 4: Overview of the proposed ST-MoE hardware platform, which consists of four key components: the Expert Prediction Unit (EPU) for algorithm-level prediction strategy, the router for gating network, the unified computing engine for key matrix computations, and the Expert Mapping Unit (EMU) for workload distribution.

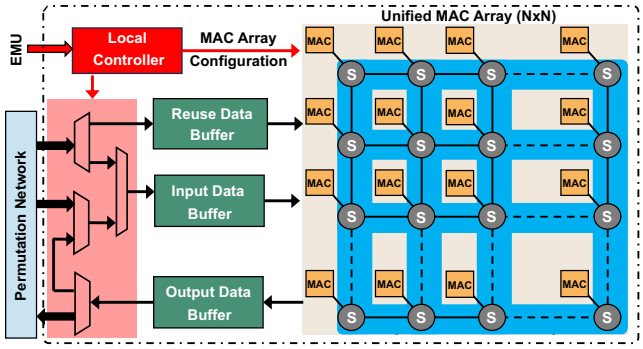

Figure 5: Architecture overview of the Processing Element (PE). It includes two input buffers for input token-related data and weight data, respectively, an NxN MAC array for matrix-matrix multiplication, and a dedicated output buffer for storing intermediate and final results. The MUX and DeMUX units are controlled by the local controller to manage local data movement.

predicted expert indices are then sent to the memory interface, which proactively fetches the corresponding expert weights from off-chip memory and stages them in the Expert/KV Buffer. When the actual gating result of a MoE layer becomes available, the EMU dynamically maps the selected experts to available PEs and configures the permutation network to route the selected expert weights and token activations from the on-chip buffers to the assigned PEs. The PEs, permutation network, and activation unit form the reconfigurable compute engine, where each PE can configure its local buffer usage and dataflow according to the reuse opportunity of the current expert computation.

4.3.2 ***Prefetching Hardware:*** To support the proposed dynamic expert prefetching strategy, ST-MoE uses the Router, the Expert Prediction Unit (EPU), the Expert Mapping Unit (EMU), the memory interface, and on-chip buffers. The router includes an $M \times K$ MAC array to execute the matrix multiplication in the gating network and generate the expert selection patterns for the input tokens, where M denotes the input batch size and K denotes the number of selected experts under the $Top\text{-}K$ routing strategy of the target LLM. The EPU uses a small local buffer to store the CCT and HT, and performs lightweight confidence-score computation based on the current routing result to predict the experts likely to be selected in the next MoE layer. The EPU also supports runtime updates to the prediction-related tables, allowing the prediction behavior to adapt to evolving expert activation patterns. The predicted expert indices are forwarded to the memory interface, which proactively fetches the corresponding expert weights from off-chip memory and stages them in the Expert/KV Buffer. In this way, expert prefetching is driven by runtime prediction rather than by a static schedule. After the actual gating result becomes available, the EMU translates the dynamic expert selection into an executable mapping. Specifically, the EMU assigns the selected experts to available PEs based on the current expert requests and the availability of prefetched experts in the on-chip buffer. It then configures the permutation network to route the selected expert weights and token activations to the assigned PEs. This mapping and routing process provides runtime reconfiguration for irregular MoE expert activation patterns. Together, these components enable efficient coordination of expert prediction, expert staging, expert-to-PE mapping, and data delivery during MoE execution.

4.3.3 ***Computation Hardware:*** ST-MoE implements a central compute array composed of N Processing Elements (PEs) to execute the main computations of each MoE layer, such as matrix multiplication. Fig. 5 shows the detailed architecture of a PE, which includes a local controller, a unified MAC array, a reuse data buffer, an input data buffer, an output data buffer, and MUX and DeMUX units. Each PE serves as a reconfigurable compute unit whose local buffer usage, MUX/DeMUX paths, and dataflow can be configured according to the workload assigned by the EMU.

After receiving the workload assignment from the EMU, the local controller configures the PE according to the dimensions of the input activations and the expert weight matrix. The controller first determines the most suitable dataflow for the current computation, such as input stationary or weight stationary, depending on which operand offers higher reuse opportunity under the assigned workload. Based on the selected dataflow, the controller configures the reuse data buffer and input data buffer so that the appropriate data can remain locally in the MAC array and be reused across multiple computation cycles. This PE-level reconfiguration does not rely on a new standalone systolic dataflow; instead, it allows the compute engine to adapt conventional dataflows to the dynamic expert assignments and prefetched expert availability produced by ST-MoE.

The local controller then configures the internal switches of the unified MAC array to match the selected dataflow. To efficiently support the dense matrix multiplication required by each MoE layer, the MAC units are organized in a systolic manner, allowing partial sums and reused operands to propagate through neighboring MAC units in a regular pattern. Such an organization improves throughput and reduces fine-grained control overhead, making it well-suited for the repeated matrix computations involved in

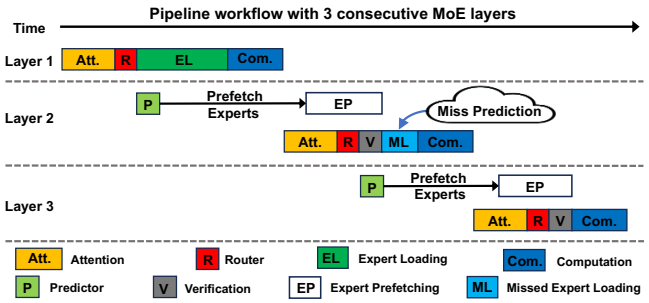

Figure 6: Pipeline workflow of three consecutive MoE layers. *Att.* is the attention operation, *R* is gating network, *EL* is expert loading, *Com.* is expert computation, *P* predicts the experts required by the subsequent layer, *V* performs verification and table update, *EP* is expert prefetching, and *ML* indicates missed loading due to expert misprediction. The figure illustrates how ST-MoE overlaps expert prefetching with attention computation while handling mispredicted experts through verification.

MoE layers. After computation, the generated intermediate or final outputs are stored in the output data buffer. When these outputs are required by the subsequent layer without additional off-chip transfer, the PE can directly forward the buffered results back to the input data buffer through the local MUX and DeMUX units. This mechanism allows the PE to reuse locally produced data in the next round of computation, thus reducing unnecessary buffer-to-memory traffic and shortening the data delivery path between consecutive computations.

4.3.4 ***Pipelined Workflow:*** ST-MoE organizes MoE inference as a pipelined workflow that overlaps expert prediction, prefetching, execution, and verification across adjacent MoE layers. Typically, each MoE layer consists of four main stages: attention, gating, expert loading, and expert computation. For ST-MoE, rather than waiting until the next layer requires expert loading, it initiates prediction and prefetching early so that expert transfer can proceed in parallel with ongoing computation. To illustrate the pipelined execution flow, we use three consecutive MoE layers as an example, as shown in Fig. 6. After receiving the result from the gating network, Layer 1 proceeds to expert loading, followed by expert computation. Simultaneously, ST-MoE uses the gating result of Layer 1 together with the CCT and HT loaded from the off-chip memory to predict the experts likely to be required by the following Layer 2. Once the prediction is completed, the predicted expert indices are temporarily buffered in the EPU, since Layer 2 must still go through its attention stage before expert prefetching is issued. When Layer 2 approaches the end of attention, the predicted expert set is sent from the EPU buffer to the prefetching hardware, which transfers the corresponding experts from off-chip memory into the on-chip buffers before Layer 2 begins expert computation. In this way, expert transfer for Layer 2 is overlapped with the computation of the attention block.

When execution advances to Layer 2, its attention and gating stages are first completed as usual, while the required experts are still being prefetched. Once the gating result of Layer 2 becomes available, ST-MoE verifies the set of prefetched experts against the actual selected experts. If some experts required by Layer 2 are not included in the prefetched set due to misprediction, as shown in Fig. 6, the missing experts are fetched immediately to preserve inference correctness. At the same time, the verification outcome of Layer 2 is used to update the HT with the latest $Top-K$ gating result and to adjust the corresponding confidence scores in the CCT. Meanwhile, the gating result of Layer 2 is reused by the EPU to trigger the next round of expert prediction for Layer 3. The same procedure is then repeated across subsequent layers, thereby forming a continuous pipelined workflow for expert prediction, transfer, execution, and update. By overlapping expert prediction and prefetching with the computation of the current layer, ST-MoE hides a substantial portion of expert loading latency. Meanwhile, the online verification and table updates ensure that prediction errors do not affect inference correctness, allowing ST-MoE to improve efficiency without compromising inference accuracy.

Table 1: Detailed MoE-based LLM models

Series	Model	#Layers	#Experts	#Top-K
Qwen	Qwen 1.5	24	60	Top-4
	Qwen 2.0	28	64	Top-6
DeepSeek	DeepSeek V2	60	160	Top-8
	DeepSeek MoE	60	128	Top-8

Table 2: Dataset statistics for profiling and evaluation (P and E denotes profiling and evaluation)

Dataset	Task	Samples (P / E)	# Decoded Tokens
CNN/DM	Article Summarization	35,000 / 145,000	~19.8 M
MATH	Math Reasoning	1,500 / 6,500	~1.2 M
HumanEval	Code Generation	30 / 134	~0.05 M

5 Evaluation and Analysis

5.1 Evaluation Setup

Evaluation Design. We extend SCALE-Sim [35] with a cycle-accurate hardware simulator implemented in C++ to model the execution behavior of the proposed ST-MoE framework. To obtain execution time, the simulator counts the number of on-chip memory access and the arithmetic operations performed by each PE. The number of on-chip memory access is used to estimate the on-chip communication time, while the number of arithmetic operations is used to calculate the computation time. The off-chip communication time is evaluated using DRAMsim2 [34]. The overall execution time is evaluated through a comprehensive analysis of temporal overlaps across different memory hierarchies, integrating the durations of on-chip communication, off-chip communication, and computational phases. During data transfer, the system leverages buffering techniques to enable concurrent activities, such as simultaneously loading the next tile from off-chip memory while distributing the current on-chip tile to PEs and executing computational tasks. The simulator models these parallel activities by determining the maximum duration across communication and computation stages,

Table 3: Key hardware components of ST-MoE and PE

ST-MoE	Value	Area (mm^2)	Power (W)
Processing Element Array	1x8	426.1	50.6
Expert/KV Buffer	16 MB	131.1	4.3
Activation Buffer	4 MB	32.8	1.1
Expert Prediction Unit	-	0.1	0.02
Router	-	28.7	5.5
DRAM	256 GB/s	-	-
PE	**Value**	**Area (mm^2)**	**Power (mW)**
Reuse Data Buffer	2 MB	16.4	0.5
Input & Output Buffer	512 KB	4.1	0.1
Unified MAC Array	64 x 64	28.6	5.8

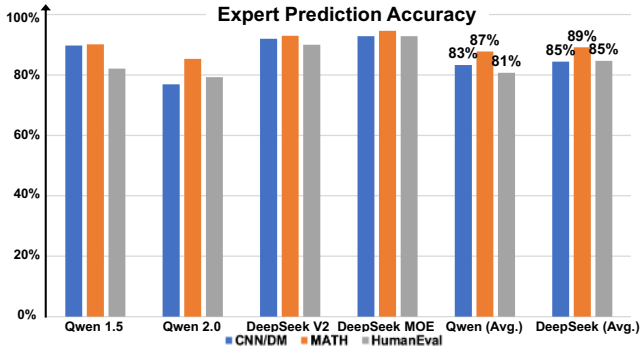

Figure 7: Prediction accuracy of the proposed dynamic expert prefetching strategy across different LLMs and applications.

thereby providing a precise result. Additionally, the simulator calculates the energy consumption by multiplying the total power consumption with the total execution time, providing a comprehensive assessment of the architecture's energetic performance. For area consumption, we implement all proposed hardware-related logic through the Synopsys Design Compiler with the TSMC 40nm library for the synthesis. We set the clock frequency at 1 GHz. We use Cacti 6.0 [28] to estimate the area, power, and access latency of all types of on-chip buffers.

Hardware Configuration. For the hardware platform, we implement the proposed ST-MoE with K processing elements (PEs), where each PE serves as an independent compute unit for a specific selected expert during each MoE layer. In our evaluation, we set $K = 8$. Each PE integrates a (64 × 64) MAC array. All MAC units are interconnected into a systolic manner to effectively perform matrix multiplication, resulting in a total of 32,768 MAC units throughout the system. In addition, we conduct an ablation study in Section 5.6 to evaluate the impact of varying the size of the MAC array within each PE on overall performance. The routing unit includes a (512 × 8) MAC array to execute the gating network, where 512 denotes the input batch size of the LLM model, and 8 corresponds to the number of PEs in the hardware platform. All MAC units use bfloat16 (BF16) arithmetic.

Benchmarks. We evaluate the proposed ST-MoE using two representative set of MoE-based LLMs, Qwen and DeepSeek, which differ in model scale and expert routing characteristics. As summarized in Table-1, these models vary in the number of layers, the number of experts per MoE layer, and the *Top-K* routing policy, providing a diverse evaluation space for analyzing ST-MoE under different expert activation patterns and model configurations. To cover representative real-world workloads, we use applications from article summarization, math reasoning, and code generation, including CNN Daily Mail (CNN/DM), MATH, and HumanEval. For each workload, a part of input samples is used for the profiling stage to construct the CCT, while the remaining samples are used to evaluate the prediction accuracy of the proposed dynamic expert prediction strategy, the total execution time, and the total energy consumption, as shown in Table-2. Token counts are estimated using the Qwen1.5 tokenizer as a unified reference.

Baselines. To perform cross-platform comparisons, we evaluate the proposed ST-MoE framework against both a conventional GPU platform and prior customized hardware designs. Specifically, we use the NVIDIA A100 GPU as the general-purpose baseline, denoted as PyGT-GPU. Additionally, we compare the proposed ST-MoE with two recent representative hardware accelerators for MoE-based LLM inference, namely Adap-gating and Pre-gated MoE, denoted as Adap-G and Pre-gated, respectively. To ensure a fair comparison, we normalize the hardware resources of all customized accelerators to match the proposed ST-MoE architecture in terms of total MAC count and off-chip memory bandwidth. In this way, the compared designs operate under the same compute and memory resource budget. For the GPU baseline, we report the performance obtained from the original platform configuration, since its architecture is fixed and cannot be scaled in the same manner as customized accelerators. This evaluation methodology ensures that the observed differences among customized designs primarily reflect architectural efficiency, rather than the advantages of raw hardware capacity.

5.2 Area and Power Analysis

To evaluate the area and power consumption of the proposed ST-MoE hardware design, we use Synopsys Design Compiler with the TSMC 40nm standard cell library. Table 3 summarizes the area and power consumption of the key hardware components in the proposed ST-MoE and PEs. Overall, the hardware cost is dominated by the compute and storage components, including the PE array, the expert/KV buffer, and the activation buffer. In particular, the PE array accounts for the largest share of both area (66%) and power consumption (81%), while the expert/KV buffer also contributes a noticeable area cost due to the storage capacity required for expert staging and KV data buffering. In contrast, the hardware overhead introduced by the prediction and prefetching support is minimal. Specifically, in our design, the EPU stores a 256-entry CCT, where each entry contains 8 candidates of 10 bits, including 8 bits for the expert candidate index and 2 bits for the confidence score. In addition, the HT contains 8 entries, each with 10 bits. Together with a small number of additional MAC units for confidence score accumulation during prediction, the proposed EPU incurs only about 0.02% area overhead, while its power overhead is nearly negligible relative to the total power consumption. These results indicate that the proposed dynamic expert prediction strategy can be implemented with minimal hardware overhead, making ST-MoE

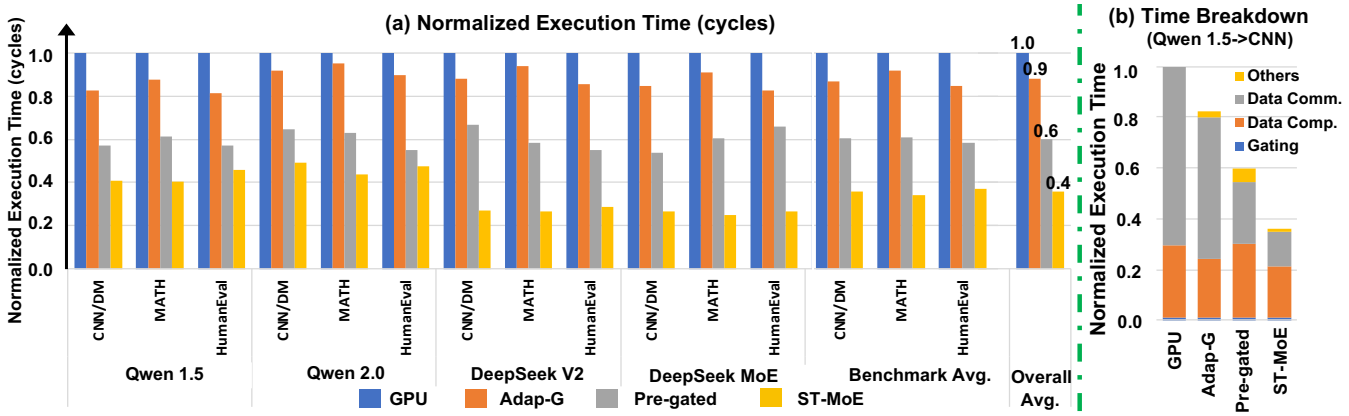

Figure 8: (a) Normalized execution time (cycles) of ST-MoE compared to state-of-the-art baselines across diverse LLMs and real-world applications. (b) Normalized execution time breakdown for Qwen1.5 on CNN/DM in the article summarization task. All results are normalized to the GPU baseline, and lower is better.

a practical design for improving the inference efficiency of MoE-based LLMs.

5.3 Expert Prediction Accuracy

Fig. 7 shows the accuracy of the proposed dynamic expert prediction strategy for different MoE-based LLMs and real-world applications. In general, ST-MoE achieves high prediction accuracy on all evaluated benchmarks, with most cases exceeding 80%, demonstrating that future expert requests can be effectively predicted using the proposed lightweight table-based strategy. A clear trend is that MATH consistently achieves higher prediction accuracy than the other applications. This is because mathematical reasoning typically follows more structured and constrained decoding patterns, leading to more stable token-level dependencies and more consistent expert activation behavior across both consecutive layers and decoding steps. In contrast, applications such as general article summarization on CNN/DM dataset often exhibit more diverse token transitions and less regular routing behavior, making expert prediction relatively more challenging. Despite these variations, ST-MoE maintains a sufficiently high accuracy across all cases, providing a strong foundation for proactive expert prefetching and for overlapping expert loading with computation.

5.4 End-to-End Execution Time:

Fig. 8(a) shows the normalized end-to-end execution time (cycles) of ST-MoE and the baseline designs across different MoE-based LLMs and real-world applications. Overall, ST-MoE consistently achieves the lowest execution time in all evaluated benchmarks, demonstrating the effectiveness of the proposed prediction-guided expert prefetching framework. On average, ST-MoE reduces the overall execution time by 60%, 56%, and 33% compared to GPU, Adap-gating, and Pre-gated MoE, respectively. By predicting future expert requests and prefetching the corresponding experts before they are needed, ST-MoE overlaps expert transfer with ongoing computation, thereby reducing the effective latency of off-chip expert memory access.

The large gap between ST-MoE and the conventional GPU baseline highlights the cost of dynamic expert loading under a general-purpose execution model. On conventional GPUs, expert weights are fetched on demand for each MoE layer, which incurs substantial data movement. As a result, expert access remains a major bottleneck and leads to the longest execution time among all LLM models. Although Adap-G improves efficiency by dynamically reducing the number of activated experts in each MoE layer instead of using a fixed number, its benefit depends on suppressing expert usage. ST-MoE instead retains the full routing outcome and reduces execution time through prediction-guided prefetching, avoiding the efficiency accuracy trade-off introduced by adaptive expert suppression. Compared to Pre-gated MoE, the advantage of ST-MoE lies in how future expert requests are predicted. Pre-gated MoE depends on an additional trained pre-gating function, whereas ST-MoE directly leverages the cross-layer and cross-token correlation to make the prediction. This allows ST-MoE to achieve lower execution time without additional training overhead or deployment complexity. Although ST-MoE still has the potential to mispredict experts, the resulting miss penalty is bounded by the high prediction accuracy shown in Fig. 7. Fig. 8(b) also shows a time breakdown for Qwen 1.5 on the CNN/DM dataset, illustrating the contribution of each component, including the gating network, computation, and data communication, to the overall execution time reduction. Clearly, the majority of the time reduction comes from lower data communication overhead, particularly reduced expert transfer.

5.5 End-to-End Energy Consumption:

Fig. 9 shows the normalized energy consumption (pJ) of ST-MoE and the baseline designs across different MoE-based LLMs and real-world applications. Compared to the conventional GPU baseline, ST-MoE incurs a modest average energy overhead of 10%. This overhead is primarily caused by prediction mismatches. When the prefetched experts do not fully match the actual routing result after performing the gating network of the current MoE layer, ST-MoE issues additional off-chip memory access to fetch those missing experts, which increases total energy consumption while preserving

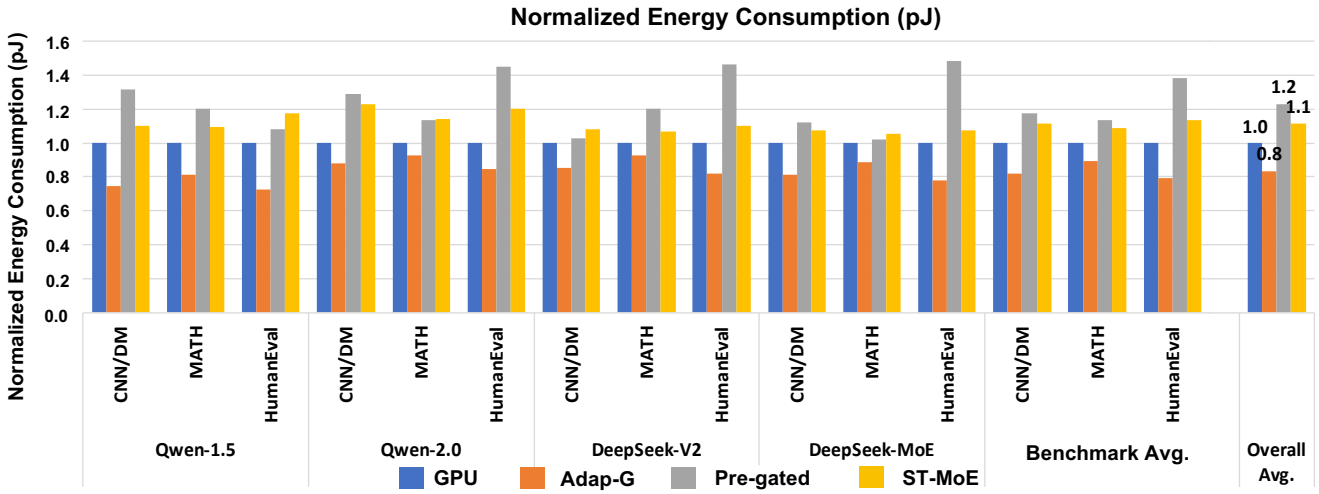

Figure 9: Normalized energy consumption (pJ) of ST-MoE compared to state-of-the-art baselines across diverse LLMs and real-world applications. The energy overhead of ST-MoE is mainly caused by the miss penalty of the expert prediction strategy. All results are normalized to the GPU baseline, and lower values indicate better energy efficiency.

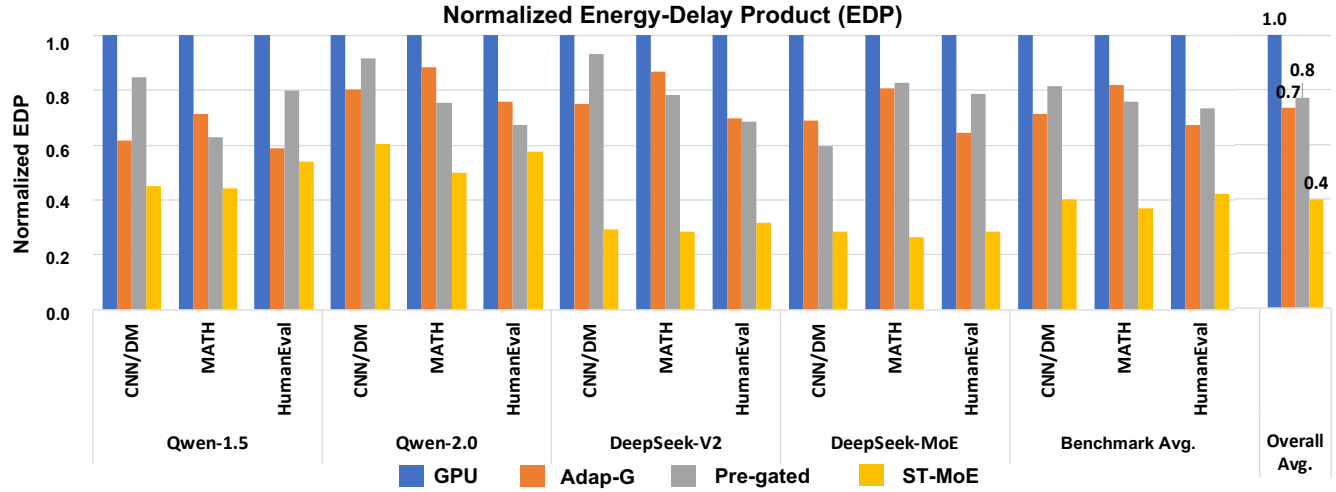

Figure 10: Normalized energy-delay product (EDP) of ST-MoE versus state-of-the-art baselines across diverse LLMs and real-world applications. All results are normalized to the GPU baseline. (lower is better)

inference accuracy. In contrast, Adap-G consumes less energy than the conventional GPU baseline because it dynamically lowers the effective $Top - K$ value and thus the number of activated experts, and the associated memory access and energy consumption. Pre-gated MoE also incurs additional energy overhead because it relies on an extra pre-gating function and proactive expert transfer, both of which introduce extra computation and data movement beyond conventional MoE execution.

Energy-Delay Product (EDP): To better capture overall energy efficiency, we further evaluate the Energy-Delay Product (EDP), which jointly accounts for execution time and energy consumption, as shown in Fig. 10. Because ST-MoE substantially reduces execution time while introducing only a modest energy overhead, EDP serves as a more comprehensive metric for evaluating the practical efficiency of the proposed framework. Across different LLM models and real-world applications, ST-MoE improves EDP by 2.5x, 1.8x, and 2.0x, compared to GPU, Adap-G, and Pre-gated MoE, respectively.

5.6 Ablation Study

To quantitatively evaluate the contribution of each proposed design component, we conduct an ablation study using Qwen across all evaluated applications. Fig. 11 details the five ST-MoE configurations considered in this study, while Fig. 12 presents the average normalized speedup, with all results normalized to the hardware-only baseline using the weight stationary (WS) dataflow. Starting from

	Fixed Dataflow	Dynamic Dataflow	HT-based Prediction	CCT-based Prediction	Dynamic Prediction
ST-MoE-1	✓				
ST-MoE-2		✓			
ST-MoE-3		✓	✓		
ST-MoE-4		✓		✓	
ST-MoE-5		✓			✓

Figure 11: Five different ST-MoE configurations for the ablation study.

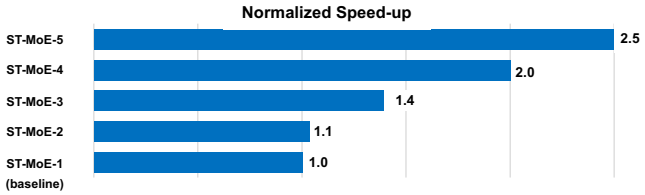

Figure 12: An ablation study of ST-MoE with five distinct configurations. All values are normalized to ST-MoE-1.

this baseline, enabling dynamic dataflow improves performance by allowing the compute hardware to adapt to different workload characteristics and improve local data reuse efficiency. Adding temporal correlation provides additional improvement, showing that expert selection information from previous decoding tokens complements the cross-layer prediction. Finally, incorporating spatial correlation further improves performance by enabling cross-layer expert prediction and proactive expert prefetching. By combining the hardware support, joint spatial-temporal prediction, and dynamic dataflow selection, the full ST-MoE design achieves the highest speedup among all evaluated configurations.

5.7 Hardware Sensitivity Analysis

To evaluate the impact of key hardware resources on ST-MoE, we conduct a hardware sensitivity analysis by varying the MAC array size within each PE and the off-chip memory bandwidth, which directly affect compute capability and expert transfer efficiency, respectively. Fig. 13 shows the normalized execution time under different hardware configurations. All results are normalized to the design with a 64×64 MAC array and 256 GB/s off-chip memory bandwidth. As the MAC array size increases, each PE can process more token computations in parallel, which reduces execution time. However, the performance gain gradually saturates at larger MAC array sizes, as the communication overhead among MAC units becomes increasingly significant. Similarly, increasing off-chip memory bandwidth reduces execution time by accelerating expert transfer, thereby improving the overlap between expert loading and computation. This benefit, however, also gradually diminishes as expert transfer ceases to be the dominant bottleneck. Overall, these results show that ST-MoE scales effectively with both compute capability and memory bandwidth, while delivering the greatest benefit.

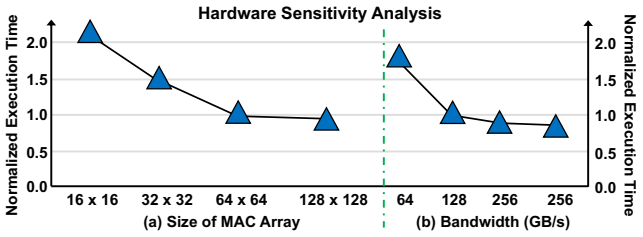

Figure 13: Hardware sensitivity analysis of ST-MoE: (a) normalized execution time under varying MAC array sizes within each PE; (b) normalized execution time under varying memory bandwidths.

6 Related Work

Efficient MoE-based LLM Inference. Recent work has explored several directions for improving the inference efficiency of MoE-based LLMs. Specifically, Adap-gating [24] reduces end-to-end inference latency by dynamically adjusting the number of activated experts in each MoE layer. However, this approach modifies the routing behavior of the original model and therefore affects model accuracy. Pre-gated MoE [19] modifies the role of the gate function by using the gate in the current MoE block to preemptively select the experts for the next MoE block, thereby moving the expert-selection dependency across adjacent blocks and enabling proactive expert transfer. However, this approach requires an additional training procedure for the pre-gated function, and changes how the original gating mechanism is used during inference. Similarly, SiDA [10] and PreScope [45] rely on additional trained predictors to improve expert prediction quality, but introduce extra training overhead and limited portability. MoE-Infinity [43] exploits skewed expert reuse patterns during decoding to guide expert caching. However, its optimization is centered on retaining frequently reused experts in the cache, whereas ST-MoE proactively stages future experts by modeling correlations across layers and decoding tokens. Fate [12] proposes a cross-layer gate mechanism that clones the current gate input to the CPU and runs the next-layer gate in parallel to generate expert prefetching decisions. By relying on CPU-side gate execution and CPU-GPU coordination, Fate is primarily suitable for CPU-GPU offloading systems. Overall, prior work improves the efficiency of MoE-based LLMs by modifying routing behavior, training auxiliary predictors, expert caching, or CPU-side gate execution. In contrast, the ST-MoE prefetching framework described in this paper jointly exploits cross-layer and consecutive-token expert activation correlations, together with a lightweight prediction strategy, to enable training-free and gate-preserving expert prefetching with reconfigurable architecture support.

LLM Accelerator Design. A large body of prior work has proposed customized hardware architectures for accelerating Transformer and LLM inference through optimized compute engines, memory hierarchies, and interconnection fabrics [6, 22, 23, 40, 42]. Specifically, H^2LLM [23] proposes a heterogeneous accelerator based on hybrid-bonding for low-batch LLM inference and performs hardware–dataflow co-exploration to balance computational capacity and bandwidth while optimizing both prefill and decoding execution. The works in [40, 42] adopt wafer-scale hardware

designs with flexible interconnection fabrics to efficiently support LLM inference. BitMoD [6] introduces fine-grained data type adaptation, which assigns different numerical data types to groups of weights to enable very low precision quantization while preserving accuracy. To support this flexibility, it employs bit-serial processing elements that efficiently handle multiple precisions and data types, although this design may introduce additional adaptation and control overhead. Despite their effectiveness, none of these customized LLM accelerators is specifically tailored for MoE layer execution during LLM inference, although MoE layers contribute a large portion of the overall execution time. Consequently, they do not directly address the core challenges of dynamically predicting, staging, and executing experts, which are critical to efficient MoE-based LLM inference.

Matrix-matrix Multiplication Design. Since the major computation pattern in each MoE layer is matrix-matrix multiplication, another relevant line of research is hardware acceleration for matrix multiplication, including sparse-dense matrix multiplication (SpMM), general matrix multiplication (GeMM), and sparse matrix–vector multiplication (SpMV) [14, 30, 32, 37, 38]. Extensor [14] is designed to accelerate sparse tensor algebra. OuterSPACE [30] and MatRaptor [37] explore different loop unrolling techniques to improve SpMM efficiency. SIGMA [32] and Tensaurus [38] also focus primarily on accelerating SpMM operations. However, most state-of-the-art matrix multiplication accelerators are optimized for sparse workloads, whereas MoE layers are dominated by dense matrix multiplication. Directly applying these prior designs to MoE inference can therefore lead to inefficient execution, as they do not fully exploit the data parallelism of dense matrix computation. Although conventional Tensor Processing Units (TPUs) are specifically designed for dense matrix multiplication, they still leave two important challenges unexplored for MoE inference. First, they lack dedicated hardware support to leverage spatial and temporal correlation for dynamic expert prediction and prefetching. Second, they do not explicitly exploit expert reuse patterns across different input tokens, which may result in frequent access to the same experts and cause substantial redundant memory traffic.

Overall, to the best of our knowledge, ST-MoE is the first expert prefetching framework that implements a lightweight prediction strategy based on spatial cross-layer correlations and temporal consecutive-token correlations among active experts. By using the predictions to proactively stage experts on chip and execute them through reconfigurable hardware support, ST-MoE hides expert memory access latency and significantly improves both performance and energy efficiency compared to prior work.

7 Conclusions

In this paper, we analyze expert selection behavior in MoE-based LLMs across diverse real-world applications and observe that expert activations exhibit strong spatial correlation across adjacent layers and temporal correlation across consecutive decoding tokens. Based on this observation, we propose ST-MoE, a framework that jointly leverages spatial and temporal correlations in expert activations to enable prediction-guided expert prefetching, thereby overlapping expert loading with computation. ST-MoE combines a lightweight dynamic expert prediction strategy with a reconfigurable hardware platform to improve performance and energy efficiency without compromising inference accuracy. Experimental results show that ST-MoE achieves 85% expert prediction accuracy and delivers average speedups of 2.5x, 2.2x, and 1.5x, along with average energy efficiency improvements of 2.5x, 1.8x, and 2.0x, compared to GPU, Adap-Gating, and Pre-gated MoE, respectively.

References

[1] Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan, Wenbin Ge, Yu Han, Fei Huang, et al. 2023. Qwen technical report. *arXiv preprint arXiv:2309.16609* (2023).

[2] Xiao Bi, Deli Chen, Guanting Chen, Shanhuang Chen, Damai Dai, Chengqi Deng, Honghui Ding, Kai Dong, Qiushi Du, Zhe Fu, et al. 2024. Deepseek llm: Scaling open-source language models with longtermism. *arXiv preprint arXiv:2401.02954* (2024).

[3] Federico Cassano, John Gouwar, Daniel Nguyen, Sydney Nguyen, Luna Phipps-Costin, Donald Pinckney, Ming-Ho Yee, Yangtian Zi, Carolyn Jane Anderson, Molly Q Feldman, et al. 2023. Multipl-e: A scalable and polyglot approach to benchmarking neural code generation. *IEEE Transactions on Software Engineering* 49, 7 (2023), 3675–3691.

[4] Danqi Chen, Jason Bolton, and Christopher D Manning. 2016. A thorough examination of the cnn/daily mail reading comprehension task. In *Proceedings of the 54th annual meeting of the association for computational linguistics (volume 1: Long papers)*. 2358–2367.

[5] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).

[6] Yuzong Chen, Ahmed F AbouElhamayed, Xilai Dai, Yang Wang, Marta Andronic, George A Constantinides, and Mohamed S Abdelfattah. 2025. Bitmod: Bit-serial mixture-of-datatype llm acceleration. In *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 1082–1097.

[7] Damai Dai, Chengqi Deng, Chenggang Zhao, RX Xu, Huazuo Gao, Deli Chen, Jiashi Li, Wangding Zeng, Xingkai Yu, Yu Wu, et al. 2024. Deepseekmoe: Towards ultimate expert specialization in mixture-of-experts language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 1280–1297.

[8] Damai Dai, Chengqi Deng, Chenggang Zhao, R.x. Xu, Huazuo Gao, Deli Chen, Jiashi Li, Wangding Zeng, Xingkai Yu, Y. Wu, Zhenda Xie, Y.k. Li, Panpan Huang, Fuli Luo, Chong Ruan, Zhifang Sui, and Wenfeng Liang. 2024. DeepSeekMoE: Towards Ultimate Expert Specialization in Mixture-of-Experts Language Models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, Bangkok, Thailand, 1280–1297. https://doi.org/10.18653/v1/2024.acl-long.70

[9] Nan Du, Yanping Huang, Andrew M Dai, Simon Tong, Dmitry Lepikhin, Yuanzhong Xu, Maxim Krikun, Yanqi Zhou, Adams Wei Yu, Orhan Firat, et al. 2022. Glam: Efficient scaling of language models with mixture-of-experts. In *International conference on machine learning*. PMLR, 5547–5569.

[10] Zhixu Du, Shiyu Li, Yuhao Wu, Xiangyu Jiang, Jingwei Sun, Qilin Zheng, Yongkai Wu, Ang Li, Hai H Li, and Yiran Chen. 2024. Sida: Sparsity-inspired data-aware serving for efficient and scalable large mixture-of-experts models. *Proceedings of Machine Learning and Systems* 6 (2024), 224–238.

[11] Sarah Fakhoury, Aaditya Naik, Georgios Sakkas, Saikat Chakraborty, and Shuvendu K Lahiri. 2024. Llm-based test-driven interactive code generation: User study and empirical evaluation. *IEEE Transactions on Software Engineering* 50, 9 (2024), 2254–2268.

[12] Zhiyuan Fang, Zicong Hong, Yuegui Huang, Yufeng Lyu, Wuhui Chen, Yue Yu, Fan Yu, and Zibin Zheng. 2025. Fate: Fast edge inference of mixture-of-experts models via cross-layer gate. *arXiv preprint arXiv:2502.12224* (2025).

[13] Zhibin Gou, Zhihong Shao, Yeyun Gong, Yelong Shen, Yujiu Yang, Minlie Huang, Nan Duan, and Weizhu Chen. 2023. Tora: A tool-integrated reasoning agent for mathematical problem solving. *arXiv preprint arXiv:2309.17452* (2023).

[14] Kartik Hegde, Hadi Asghari-Moghaddam, Michael Pellauer, Neal Crago, Aamer Jaleel, Edgar Solomonik, Joel Emer, and Christopher W Fletcher. 2019. Extensor: An accelerator for sparse tensor algebra. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*. 319–333.

[15] Karl Moritz Hermann, Tomas Kocisky, Edward Grefenstette, Lasse Espeholt, Will Kay, Mustafa Suleyman, and Phil Blunsom. 2015. Teaching machines to read and comprehend. *Advances in neural information processing systems* 28 (2015).

[16] Wenbo Hu, Yifan Xu, Yi Li, Weiyue Li, Zeyuan Chen, and Zhuowen Tu. 2024. Bliva: A simple multimodal llm for better handling of text-rich visual questions. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 2256–2264.

[17] Dong Huang, Jie M. Zhang, Qingwen Bu, Xiaofei Xie, Junjie Chen, and Heming Cui. 2025. Bias testing and mitigation in llm-based code generation. *ACM*

Transactions on Software Engineering and Methodology 35, 1 (2025), 1–31.

[18] Changho Hwang, Wei Cui, Yifan Xiong, Ziyue Yang, Ze Liu, Han Hu, Zilong Wang, Rafael Salas, Jithin Jose, Prabhat Ram, et al. 2023. Tutel: Adaptive mixture-of-experts at scale. *Proceedings of Machine Learning and Systems* 5 (2023), 269–287.

[19] Ranggi Hwang, Jianyu Wei, Shijie Cao, Changho Hwang, Xiaohu Tang, Ting Cao, and Mao Yang. 2024. Pre-gated moe: An algorithm-system co-design for fast and scalable mixture-of-expert inference. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 1018–1031.

[20] Albert Q Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, et al. 2024. Mixtral of experts. *arXiv preprint arXiv:2401.04088* (2024).

[21] Callie Y Kim, Christine P Lee, and Bilge Mutlu. 2024. Understanding large-language model (llm)-powered human-robot interaction. In *Proceedings of the 2024 ACM/IEEE international conference on human-robot interaction*. 371–380.

[22] Hyojung Lee, Daehyeon Baek, Jimyoung Son, Jieun Choi, Kihyo Moon, and Minsung Jang. 2025. Paise: Pim-accelerated inference scheduling engine for transformer-based llm. In *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 1707–1719.

[23] Cong Li, Yihan Yin, Xintong Wu, Jingchen Zhu, Zhutianya Gao, Dimin Niu, Qiang Wu, Xin Si, Yuan Xie, Chen Zhang, et al. 2025. H2-llm: Hardware-dataflow co-exploration for heterogeneous hybrid-bonding-based low-batch llm inference. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture*. 194–210.

[24] Jiamin Li, Qiang Su, Yitao Yang, Yimin Jiang, Cong Wang, and Hong Xu. 2023. Adaptive gating in mixture-of-experts based language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 3577–3587.

[25] Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. 2023. Let's Verify Step by Step. *arXiv preprint arXiv:2305.20050* (2023).

[26] Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. 2024. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437* (2024).

[27] AI Meta. 2025. The llama 4 herd: The beginning of a new era of natively multimodal ai innovation.

[28] Naveen Muralimanohar, Rajeev Balasubramonian, and Norman P Jouppi. 2009. CACTI 6.0: A tool to understand large caches. *University of Utah and Hewlett Packard Laboratories, Tech. Rep* 147 (2009).

[29] Daye Nam, Andrew Macvean, Vincent Hellendoorn, Bogdan Vasilescu, and Brad Myers. 2024. Using an llm to help with code understanding. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 1–13.

[30] Subhankar Pal, Jonathan Beaumont, Dong-Hyeon Park, Aporva Amarnath, Siying Feng, Chaitali Chakrabarti, Hun-Seok Kim, David Blaauw, Trevor Mudge, and Ronald Dreslinski. 2018. Outerspace: An outer product based sparse matrix multiplication accelerator. In *2018 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 724–736.

[31] Qiwei Peng, Yekun Chai, and Xuhong Li. 2024. Humaneval-xl: A multilingual code generation benchmark for cross-lingual natural language generalization. In *Proceedings of the 2024 joint international conference on computational linguistics, language resources and evaluation (LREC-COLING 2024)*. 8383–8394.

[32] Eric Qin, Ananda Samajdar, Hyoukjun Kwon, Vineet Nadella, Sudarshan Srinivasan, Dipankar Das, Bharat Kaul, and Tushar Krishna. 2020. Sigma: A sparse and irregular gemm accelerator with flexible interconnects for dnn training. In *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 58–70.

[33] Samyam Rajbhandari, Conglong Li, Zhewei Yao, Minjia Zhang, Reza Yazdani Aminabadi, Ammar Ahmad Awan, Jeff Rasley, and Yuxiong He. 2022. Deepspeed-moe: Advancing mixture-of-experts inference and training to power next-generation ai scale. In *International conference on machine learning*. PMLR, 18332–18346.

[34] Paul Rosenfeld, Elliott Cooper-Balis, and Bruce Jacob. 2011. DRAMSim2: A cycle accurate memory system simulator. *IEEE computer architecture letters* 10, 1 (2011), 16–19.

[35] Ananda Samajdar, Yuhao Zhu, Paul Whatmough, Matthew Mattina, and Tushar Krishna. 2018. Scale-sim: Systolic cnn accelerator simulator. *arXiv preprint arXiv:1811.02883* (2018).

[36] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. 2017. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *arXiv preprint arXiv:1701.06538* (2017).

[37] Nitish Srivastava, Hanchen Jin, Jie Liu, David Albonesi, and Zhiru Zhang. 2020. Matraptor: A sparse-sparse matrix multiplication accelerator based on row-wise product. In *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 766–780.

[38] Nitish Srivastava, Hanchen Jin, Shaden Smith, Hongbo Rong, David Albonesi, and Zhiru Zhang. 2020. Tensaurus: A versatile accelerator for mixed sparse-dense tensor computations. In *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 689–702.

[39] Peng Wang, Shuai Bai, Sinan Tan, Shijie Wang, Zhihao Fan, Jinze Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, et al. 2024. Qwen2-vl: Enhancing vision-language model's perception of the world at any resolution. *arXiv preprint arXiv:2409.12191* (2024).

[40] Taiquan Wei, Huizheng Wang, Zichuan Wang, Shouyi Yin, and Yang Hu. 2025. Spatial-Aware Orchestration of LLM Attention on Waferscale Chips. In *International Symposium on Advanced Parallel Processing Technologies*. Springer, 386–391.

[41] Shengqiong Wu, Hao Fei, Leigang Qu, Wei Ji, and Tat-Seng Chua. 2024. Next-gpt: Any-to-any multimodal llm. In *Forty-first International Conference on Machine Learning*.

[42] Zheng Xu, Dehao Kong, Jiaxin Liu, Jinxi Li, Jingxiang Hou, Xu Dai, Chao Li, Shaojun Wei, Yang Hu, and Shouyi Yin. 2025. Wsc-llm: Efficient llm service and architecture co-exploration for wafer-scale chips. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture*. 1–17.

[43] Leyang Xue, Yao Fu, Zhan Lu, Luo Mai, and Mahesh Marina. 2024. Moe-infinity: Efficient moe inference on personal machines with sparsity-aware expert cache. *arXiv preprint arXiv:2401.14361* (2024).

[44] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388* (2025).

[45] Enda Yu, Zhaoning Zhang, Dezun Dong, Yongwei Wu, and Xiangke Liao. 2025. PreScope: Unleashing the Power of Prefetching for Resource-Constrained MoE Inference. *arXiv preprint arXiv:2509.23638* (2025).

[46] Jun Zhan, Junqi Dai, Jiasheng Ye, Yunhua Zhou, Dong Zhang, Zhigeng Liu, Xin Zhang, Ruibin Yuan, Ge Zhang, Linyang Li, et al. 2024. Anygpt: Unified multimodal llm with discrete sequence modeling. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 9637–9662.

[47] Biao Zhang, Zhongtao Liu, Colin Cherry, and Orhan Firat. 2024. When scaling meets llm finetuning: The effect of data, model and finetuning method. *arXiv preprint arXiv:2402.17193* (2024).

[48] Yadong Zhang, Shaoguang Mao, Tao Ge, Xun Wang, Adrian de Wynter, Yan Xia, Wenshan Wu, Ting Song, Man Lan, and Furu Wei. 2024. Llm as a mastermind: A survey of strategic reasoning with large language models. *arXiv preprint arXiv:2404.01230* (2024).